GraphML-Studio

A server-less web application for Graph Machine Learning Tasks

Bitan Majumder
M.Sc. Computer Science, 2nd year
24MSCSCPY0005

Graph is a very crucial data structure that is been used in our real world extensively, often to describe complex structures between entities or events and their relations. With the emergence of Social Media and Web applications this has become important more than ever before. Fields like Social Network Analysis, Network Security, Navigation and Transportations, Recommender System design etc. require Graphs to analyze scenarios and predict different outcomes.

But it is not something that is only being used by Computer Science researchers and Engineers. The user of Graph modelling has extended to different field experts who have no technical knowledge about graphs. For example, Social Science people who use Social Media data to analyze and identify different patterns in social behaviours. For them, the social media data is an asset even though they have little knowledge of handling it.

In some cases, the graph modelling and analysis could be tedious, when it involves a huge no. of nodes. If the graph is a dense graph it could lead to a hairball structure, making it difficult to visualize. Apart from that, the analysis of graph structures often takes difficult calculations to make. The community detection, centrality measures, density measure – all these requires complex mathematics that could be beyond the scope of a person who has no mathematical background.

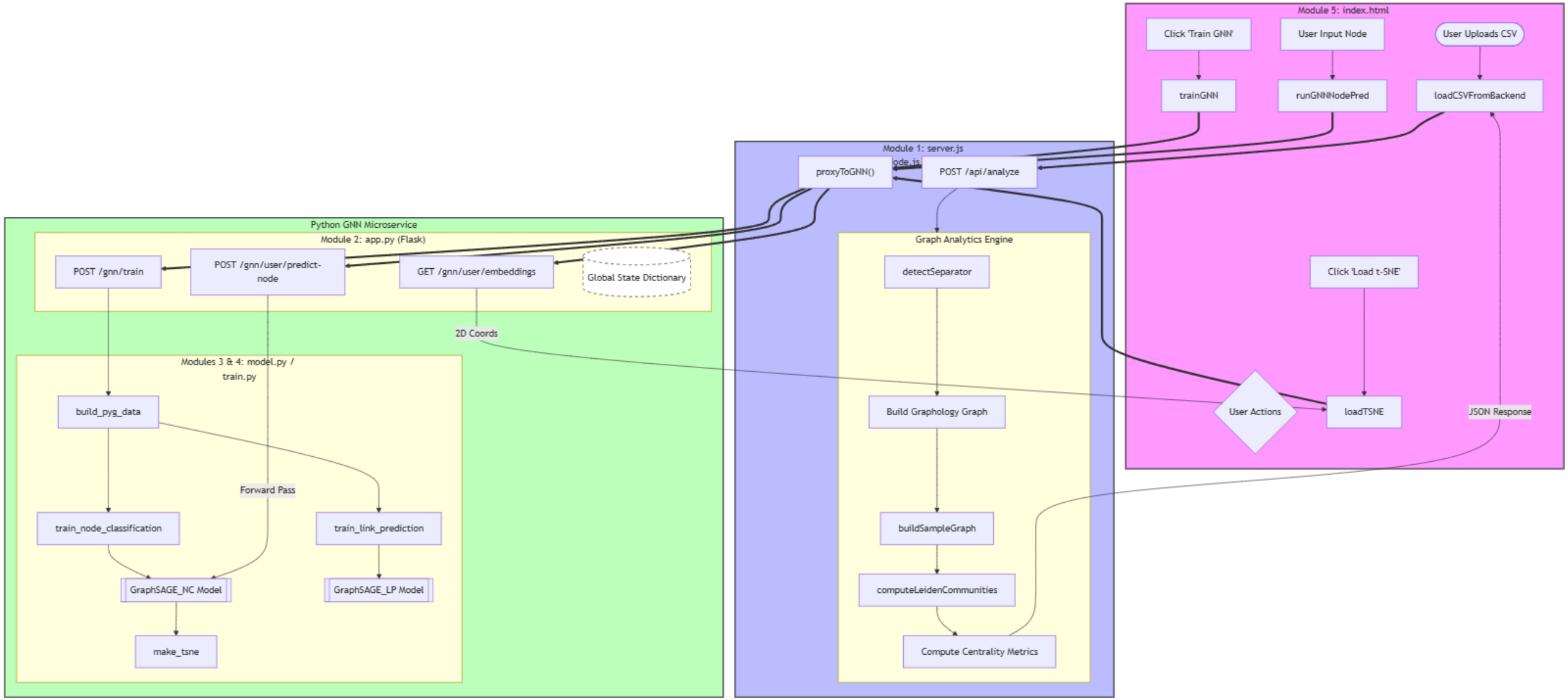
Now to tackle most of these problems a bunch of tools are available on the internet. And the most popular among them is Gephi, which is a software that handles Graph Analysis tasks very well. But Gephi also has some downsides. It has to be downloaded locally by the user and its size could vary between 115 to 150 MB depending upon the OS. Also, Gephi is for analytical tasks, but it fails to perform any prediction task which is an inherent part of Graph Machine Learning

So, this is where the GraphML-Studio comes into picture. GraphML-Studio is a server-less browser-based application that could perform both Social Network Analysis tasks along with prediction tasks. It is an integration of SNA metrics with Graph Neural Network that could handle tasks like Community Detection, Centrality Measure, node classification and Link prediction.

It is a completely free web-application that needs no download, no GPU, and no code setup. It could be accessed online and to use it the user needs to upload the CSV file of the Edge information only. It will build the graph from that CSV file. Only specification needed about the CSV is that it must have the source, the target and the weight of the edge.

The uniqueness of GraphML-Studio lies in its backend which employs both GNN and non-GNN services. The GraphSAGE model is capable of performing prediction tasks in a Graph dataset with training on only 5% of the data. An API is built on this GraphSAGE model which is being called by the node server. But here is a unique fact about the application. Even if the model is unreachable for some reasons, the node server could predict the nodes and edges. For node classification, if the GNN is not available, it uses a simple majority-vote heuristic: it looks at the communities of all specified neighbors and picks the most common one. Similarly for edge prediction, it uses a heuristic based on the Adamic-Adar idea: for each candidate node, it counts how many of the new node's known connections are also connected to that candidate (shared neighbors). This count is divided by the log of the candidate's degree to dampen the contribution of very high-degree hubs. Higher scores mean higher predicted link probability.

- The GraphML-Studio is a Server-less web-application that directly runs on browser and offers a no-code setup.
- The application could handle up to 5000 nodes and 15000 edges. The browser could not render more than this smoothly.
- For community detection it offers 2 options – Leiden Community Detection and Louvain Community Detection
- For centrality measure it offers 3 options – Betweenness centrality, Degree centrality and Eigenvector centrality
- Along with the Graph visualization it also offers a t-SNE view. t-SNE stands for t-distributed Stochastic Neighbour Embedding which is a nonlinear dimensionality reduction technique used to visualize high-dimensional datasets in 2D or 3D plots. However, this service is only available if the data is trained on the GNN.
- A community-based colouring of nodes is also possible.
- For both training cases (node classification and link prediction), Adam Optimizer (weight decay 0.0005) has been used, and Binary Cross-Entropy as Loss function.
- For the validation accuracy Receiver Operating Characteristic (ROC) curve has been used.
- For node classification, different attributes of the node like Term Frequency, Inverse Document Frequency, Pointwise Mutual Information etc. are considered as node features.


Module 5: index.html
Click 'Train GNN'
User Input Node
User Uploads CSV
trainGNN
runGNNNodePred
loadCSVFromBackend
Module 1: server.js
ode.js
proxyToGNN()
POST /api/analyze
Python GNN Microservice
Module 2: app.py (Flask)
POST /gnn/train
POST /gnn/user/predict-node
GET /gnn/user/embeddings
Global State Dictionary
Graph Analytics Engine
detectSeparator
Click 'Load t-SNE'
2D Coords
Modules 3 & 4: model.py / train.py
build_pyg_data
Build Graphology Graph
User Actions
loadTSNE
JSON Response
Forward Pass
train_node_classification
train_link_prediction
buildSampleGraph
GraphSAGE_NC Model
GraphSAGE_LP Model
computeLeidenCommunities
make_tsne
Compute Centrality Metrics

Data Flow Walkthrough

To tie everything together, here is what happens step by step when a user uploads a CSV and runs the full GNN pipeline:

#	Location	Action
1	`index.html`	User drops a CSV onto the upload zone → loadCSVFromBackend() is called
2	`index.html` → `server.js`	FormData POST to /api/analyze
3	`server.js`	detectSeparator(), looksLikeHeaderRow() parse the CSV; Graphology graph is built
4	`server.js`	buildSampleGraph() samples if too large; louvain() and computeLeidenCommunities() run
5	`server.js`	Centrality (betweenness, eigenvector, degree) computed; JSON response assembled
6	`index.html`	graphData stored; buildVisNetwork() renders the graph; task buttons enabled
7	`index.html`	User clicks "Train GNN" → trainGNN() POSTs nodes+edges to /api/gnn/train
8	`server.js`	proxyToGNN() forwards request to Python on port 5001
9	`app.py`	graph_hash() computed; cache checked; if miss: build_pyg_data() called
10	`train.py`	train_node_classification(): RandomNodeSplit, Adam optimizer, early stopping
11	`train.py`	train_link_prediction(): RandomLinkSplit, BCE loss, AUC-ROC monitoring
12	`app.py`	GraphSAGE_NC.embed() extracts 128-dim embeddings; make_tsne() projects to 2D
13	`app.py`	Model weights cached to disk; metrics returned via Flask
14	`index.html`	User clicks "Load t-SNE" → loadTSNE() fetches 2D points; renderTSNE() draws canvas
15	`index.html`	User types a node name and clicks "Predict" → runGNNNodePred() sends to Python
16	`app.py`	user_predict_node(): forward pass, softmax, top-5 probabilities returned
17	`index.html`	Result displayed with community color, confidence bar, and top-5 breakdown

Tech Stack used –

1. Node.js

Node.js is a cross-platform, open-source JavaScript runtime environment that executes JavaScript code outside a web browser, built on Chrome's V8 engine. It is renowned for its event-driven, non-blocking I/O model, making it ideal for building scalable network applications. In this project, **Node.js serves as the "Orchestrator" and Backend Gateway.** Using the Express framework, it manages the primary web server, handles large CSV file uploads, and performs initial graph processing using the *Graphology* library. It acts as a sophisticated proxy that manages UI constraints (sampling large graphs to keep the browser performant) and communicates seamlessly with the Python-based GNN microservice.

2. Flask

Flask is a micro web framework written in Python, designed to be lightweight, modular, and easy to transform into a high-performance API. It does not require particular tools or libraries, making it the perfect choice for wrapping machine learning models into accessible services. In this application, **Flask functions as the "Machine Learning API."** It hosts the GNN inference engine, providing dedicated endpoints for training custom models and generating t-SNE embeddings. By using Flask, the application separates the heavy computational load of Deep Learning from the web-handling logic of Node.js, ensuring that the GNN models remain resident in memory for rapid response times.

3. Transformers

Transformers is a state-of-the-art library by Hugging Face that provides thousands of pre-trained models to perform tasks on different modalities such as text, vision, and audio. It is built around the "Attention" mechanism, which allows models to weigh the importance of different parts of input data. In the context of this project, **Transformers are utilized for Semantic Feature Extraction.** Before the graph structure is even processed, the library is used to convert raw node labels (such as tweets from the Farmer's Protest or entity names) into high-dimensional vector embeddings. These "text-based features" provide the initial mathematical representation of the nodes, allowing the GNN to understand the meaning of the data as well as its connections.

4. Torch_Geometric (PyG)

PyTorch Geometric (PyG) is a specialized extension library built upon PyTorch to easily write and train Graph Neural Networks (GNNs) for structured data. It provides highly optimized implementations of various graph convolution layers and supports massive graphs through efficient sparse matrix operations. In this application, **PyG is the core "GNN Engine."** It is used to implement the **GraphSAGE** architecture, which performs both Node Classification (predicting communities) and Link Prediction (suggesting new connections). PyG's inductive capabilities are what allow this app to take a brand-new node and predict its behavior based solely on its neighborhood structure in real-time.

1. Community Detection: Louvain & Leiden

Definition: These are modularity-based clustering algorithms used to find "natural groups" or communities within a network by maximizing the density of edges within groups versus between them.

In this Project: These algorithms provide the **"Ground Truth" labels.** When a user uploads a graph, the system first runs the Louvain and Leiden algorithms (via the *LeidenAlgorithm* in the Node.js backend) to identify communities. These results are then used as training targets for the GraphSAGE model to learn from.

2. Network Topology: Centrality Measures

Definition: Metrics like **Degree**, **Betweenness**, and **Eigenvector Centrality** quantify the "importance" or "influence" of a node based on its connections and its position as a bridge between other clusters.

In this Project: These measures act as **Structural Features.** The application calculates these values to help the GNN understand a node's role. For example, a node with high "Betweenness" is flagged as a "Hub" or "Broker," which significantly influences the GNN's prediction of its community and potential future links.

3. Heuristic Node Classification (GNN Offline Mode)

Definition: A "Lazy Learner" approach that uses **Weighted Majority Voting** based on a node’s immediate neighbors.

In this Project: This is the **Fallback Logic.** If the GNN service is offline, the Node.js server uses a neighbor-influence algorithm. It looks at the communities of the nodes a new entry is connecting to and predicts the community based on the highest frequency and weight of those neighbor types.

4. Structural Link Prediction (GNN Offline Mode)

Definition: An algorithm based on **Common Neighbors** and **Resource Allocation** (Adamic-Adar style), which calculates the probability of a link based on shared connections.

In this Project: This serves as the **Heuristic Recommender.** Without the GNN, the system suggests new connections by looking for "triadic closures"—if Node A and Node B share many common friends, the offline algorithm predicts they should be connected, adjusted by a "Log-Degree" penalty to prevent recommending only high-degree hubs.

5. Statistical NLP: TF-IDF & PMI

Definition: TF-IDF (Term Frequency-Inverse Document Frequency) measures word importance, while **PMI** (Pointwise Mutual Information) measures the strength of association between two words.

In this Project: These are the **Semantic Anchors.** For text-based graphs (like the Farmer's Protest demo), these algorithms weight the nodes. High TF-IDF words are given more "strength" in the graph, ensuring that unique, meaningful terms have a higher impact on the GNN's learning process than common, generic words.

6. Visualization: t-SNE (T-distributed Stochastic Neighbor Embedding)

Definition: A non-linear dimensionality reduction technique that maps high-dimensional data (like 128-bit GNN embeddings) into a 2D or 3D space.

In this Project: This is the **Human-Interpretability Layer.** It takes the complex mathematical vectors generated by GraphSAGE and projects them into the X/Y coordinates seen on the user interface, allowing users to visually see "clusters" of related concepts.